MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Gabriela Pricop, FAF-241

Report

Laboratory Work No. 9

Operating Systems

Checked by:
Reițman Patricia, *university assistant*

Chișinău – 2026

Contents

Goal **3**

1 Part 1: See the calls of a real command **3**

2 Part 2: A program that calls the kernel directly **5**

Conclusion **9**

References **10**

Goal

In the lecture the door between a program and the kernel was crossed: the system call. The purpose of this laboratory work is to watch that door from the outside with strace, a program that prints every system call another program makes, together with its arguments and its result. A system call is normally invisible, because the C library hides it behind a function such as printf or fopen; strace makes it visible without changing the program that is traced and without recompiling it.

The work has three parts. The first part traces a familiar command, ls -l, in order to see the calls that a real program makes and to find which call writes to the screen and to which descriptor. The second part is a short program, syscalls.c, that opens a file, writes a line into it, reads the line back and closes the file, using the system calls open, write, read and close directly instead of the functions of the standard library; the same program is then run under strace, so that the three C calls can be compared with the lines of the trace. The third part counts the calls of one chosen kind, which shows how often a program crosses that door.

The work was done on Linux, in the directory os-lab9 of the course repository. The programs were compiled with gcc -Wall -Wextra and traced with strace, which was installed with sudo apt install strace.

Part 1: See the calls of a real command

Questions:

- Which system call does ls use to print to the screen? To which descriptor does it write?

Answers:

- It uses write, and it writes to the descriptor 1, the standard output. In the trace the line has the form write(1, "...", n) = n, where the first argument is the descriptor, the second one is the text that is handed to the kernel and the number after the equals sign is how many bytes the kernel accepted. The program does not know what is behind the descriptor 1: when the output goes to the terminal, the same call reaches the screen, and when the output is redirected with a greater-than sign, the same call reaches a file, exactly as the redirection built by hand in a previous laboratory work showed. The three standard descriptors are listed in Table 3.



Figure 1.1: Installing strace and the first lines of the trace of ls -l

sudo apt install strace – the package was already present on the system, and apt answered that strace is already the newest version, 6.19+ds-0ubuntu5, so nothing was installed, upgraded or removed. The tool works by the system call ptrace: the kernel stops the traced process at every entry into and exit from a system call, and strace prints the name, the arguments and the returned value.

strace -f -e trace=openat,read,write,close ls -l – runs ls -l under the tracer. The option -e trace= keeps only the four calls that are interesting here, out of the several hundred that exist, and -f asks the tracer to follow the children as well, if the command creates any. The first lines are not the work of ls at all: they belong to the dynamic loader, which opens /etc/ld.so.cache to find where the shared libraries are, closes it and then opens the libraries themselves, such as /usr/lib/x86_64-linux-gnu/libselinux.so.1. Each of these openat calls returns the descriptor 3, because the previous file was closed before the next one was opened, so the number 3 becomes free again every time. The flag O_RDONLY shows that the files are opened only for reading, and O_CLOEXEC asks the kernel to close them automatically if the process executes another program, so that a descriptor is not inherited by accident.

The number of calls – even a command as simple as ls -l crosses the door to the kernel a few dozen times before it prints anything: one openat, one read and one close for every library, several calls to learn the properties of the files of the directory, and only at the end the write calls of the listing. The entries of the directory themselves are not read with read but with getdents64, a call made exactly for directories, which is why it does not appear in a trace filtered on these four names. The calls are summarized in Table 1.

Call	When it appears in the trace	What it does
openat	at the start, for the cache and for every shared library	asks the kernel for a file and gets back the smallest free descriptor
read	after every openat of a library	copies bytes from the kernel into the memory of the process
close	after every library is loaded	gives the descriptor back to the kernel
write	at the end, when the listing is printed	hands the bytes of the output to the kernel, on the descriptor 1

Table 1: The four traced calls in the trace of ls -l

Table 1 shows that most of the calls of a simple command are spent on starting the process, and that the visible output of the command is only the last and the smallest part of its conversation with the kernel.

Part 2: A program that calls the kernel directly

Questions:

- The descriptor returned by the first openat is 3, not 0, 1 or 2. Why are those three already taken?

Answers:

- Because every process is started with three descriptors already open, inherited from the shell: 0 is the standard input, 1 is the standard output and 2 is the standard error, and all three point to the terminal. The kernel always gives the smallest free descriptor, so the first file opened by the program receives the number 3. The same rule was used in a previous laboratory work to build a redirection by hand: closing the descriptor 1 before open made the file take the number 1, and the program then wrote into the file without knowing it. In the trace this is visible twice, since the second openat also returns 3: the first file was closed before, so the number became free again. The three descriptors are listed in Table 3.

syscalls.c – the program asks the kernel directly for everything that concerns the file. It opens note.txt with open and the flags O_CREAT, O_WRONLY and O_TRUNC, which mean create the file if it does not exist, open it only for writing and empty it if it already has content, with the permissions 0644; it writes the message with write, which receives the descriptor, the address of the bytes and their number, and returns how many bytes the kernel accepted; it closes the file with close, opens it again with O_RDONLY, reads the content back with read into a buffer of 64 bytes, terminates the text with a zero and closes the file again. The only calls that do not concern the file are the three printf, which print the results on the screen; they belong to the C library, but they also reach the kernel in the end, as the trace shows. The source is given below.

```c
/* syscalls.c - open, write, read and close, using only system calls.
 * Build: gcc -Wall -Wextra -o syscalls syscalls.c
 * Run:   ./syscalls
 * See the calls: strace ./syscalls
 */
#include <fcntl.h>      /* open */
#include <unistd.h>     /* write, read, close */
#include <string.h>     /* strlen */
#include <stdio.h>      /* printf */
int main(void) {
    const char *msg = "hello, kernel\n";

    /* open() is a system call: ask the kernel for a file, get back a descriptor */
    int fd = open("note.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (fd < 0) { perror("open"); return 1; }
    printf("open returned descriptor %d\n", fd);

    /* write() is a system call: hand the bytes to the kernel */
    ssize_t n = write(fd, msg, strlen(msg));
    printf("write sent %zd bytes\n", n);
    close(fd);

    /* read it back, through the kernel again */
    fd = open("note.txt", O_RDONLY);
    char buf[64];
    n = read(fd, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    printf("read got %zd bytes: %s", n, buf);
    close(fd);
    return 0;
}
```

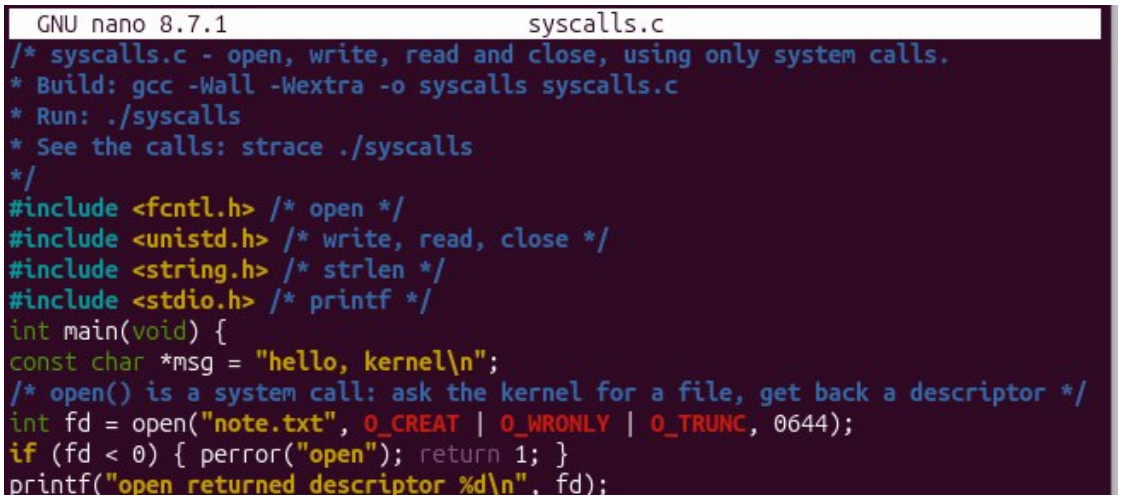


Figure 2.1: The beginning of syscalls.c in the editor

nano syscalls.c – the four headers at the top are not decoration: fcntl.h declares open and the flags, unistd.h declares write, read and close, string.h declares strlen and stdio.h declares printf. Unlike the programs of the previous laboratory work, which ran inside xv6 and had no library at all, here the system calls are reached through the thin wrappers of the C library, but the wrapper does nothing except place the arguments in the right registers and execute the instruction that enters the kernel.

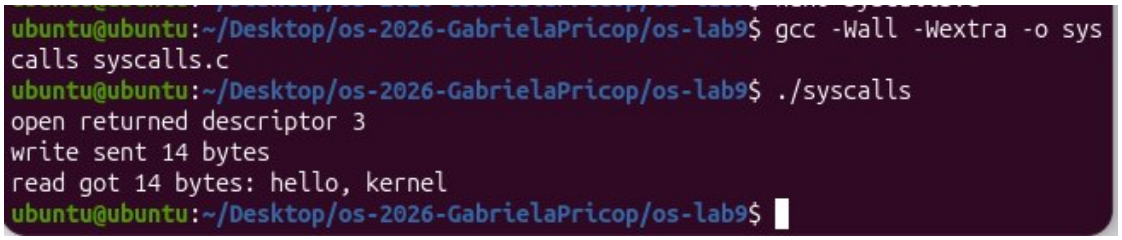


Figure 2.2: Compiling and running syscalls

gcc -Wall -Wextra -o syscalls syscalls.c ; ./syscalls – the program compiles without any warning and prints three lines: open returned descriptor 3, write sent 14 bytes and read got 14 bytes: hello, kernel. The number 14 is the length of the message, including the newline at its end, and it appears twice, once as the value returned by write and once as the value returned by read, which proves that the bytes made the whole journey into the kernel, onto the disk and back into the memory of the process.

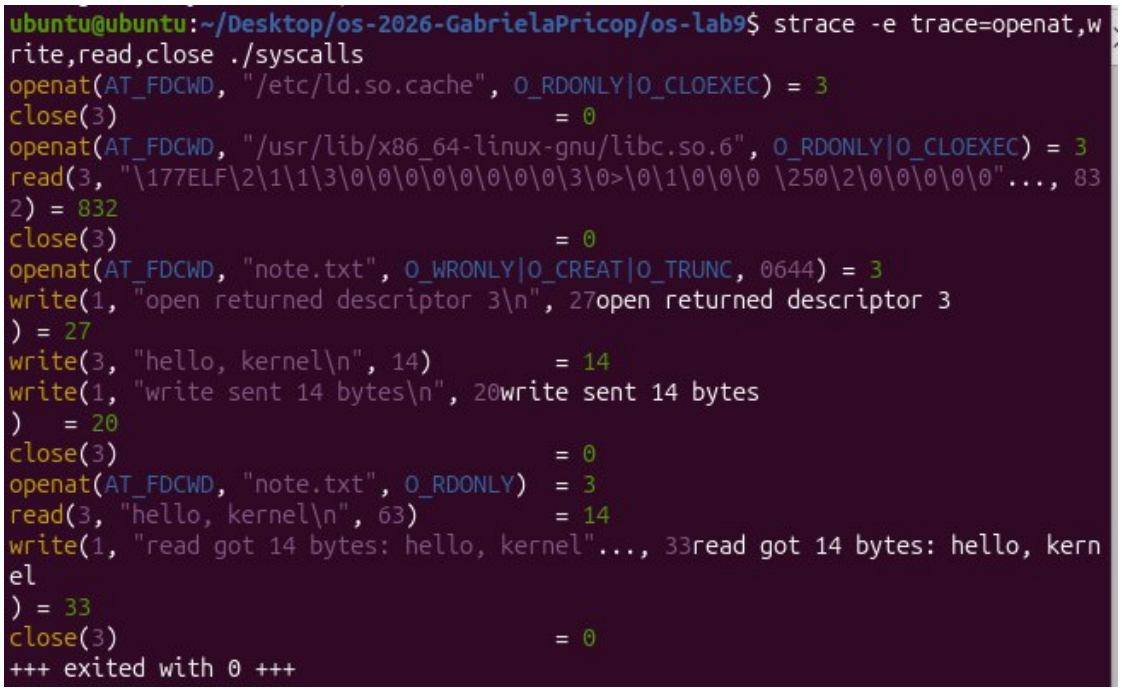


Figure 2.3: The trace of syscalls: the C calls become system calls

strace -e trace=openat,write,read,close ./syscalls – the same program is run again, this time under the tracer, and the output of the program is mixed with the lines of the trace, because the trace goes to the standard error and the program writes to the standard output. The first four lines still belong to the loader, which opens /etc/ld.so.cache and then /usr/lib/x86_64-linux-gnu/libc.so.6 and reads its first 832 bytes, the header that begins with the four characters of the ELF format. After that come the calls of the program itself, in the exact order of the source: openat with O_WRONLY|O_CREAT|O_TRUNC and 0644 returns 3, write(3, "hello, kernel\n", 14) returns 14, close(3) returns 0, openat with O_RDONLY returns 3 again, read(3, "hello, kernel\n", 63) returns 14 and close(3) returns 0. The last line, +++ exited with 0 +++, is printed by strace and not by the program, and it reports the status with which the process ended.

What the trace shows about printf – the three printf of the program also appear, as write calls on the descriptor 1: write(1, "open returned descriptor 3\n", 27), write(1, "write sent 14 bytes\n", 20) and write(1, "read got 14 bytes: hello, kern"..., 33). This is the answer to the first part seen from the other side: printf is not a system call, it is a function of the library that formats the text and then calls write, and strace sees only the call that actually crosses into the kernel. The name openat instead of open is a similar case: the library translates the older open into the more general openat, whose first argument AT_FDCWD means that the path is to be interpreted relative to the current directory.

read(3, "hello, kernel\n", 63) = 14 – the third argument of read is 63 and not 14, because the program asked for the size of its buffer minus one, which is the most it can accept, while the returned value 14 is how many bytes the file actually contained. A read that returns less than it was asked for is normal and not an error; it returns 0 only at the end of the file. The correspondence between the C calls and the traced lines is given in

Table 2.

In the C code	In the trace	The line of the trace	Result
open	openat	openat(AT_FDCWD, "note.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644)	3, the first free descriptor
write	write	write(3, "hello, kernel", 14)	14, all the bytes accepted
close	close	close(3)	0, the descriptor is free again
open	openat	openat(AT_FDCWD, "note.txt", O_RDONLY)	3, free again after close
read	read	read(3, "hello, kernel", 63)	14, the size of the file
printf	write	write(1, "open returned descriptor 3", 27)	27, printed on the screen

Table 2: Every C call of syscalls.c and the system call it becomes

Descriptor	Name	Where it points at the start	Who opened it
0	standard input	the terminal	inherited from the shell
1	standard output	the terminal	inherited from the shell
2	standard error	the terminal	inherited from the shell
3	the first free one	note.txt	opened by the program

Table 3: Why the first file of a program receives the descriptor 3

Table 2 shows that every operation on the file became exactly one system call, in the same order as in the source, and Table 3 shows that the number 3 is not chosen by the program but is simply the smallest number that the kernel had free.

Conclusion

The laboratory work made visible the border between a program and the kernel. In the first part, the trace of ls -l showed that a command which seems simple crosses that border a few dozen times: the dynamic loader opens /etc/ld.so.cache and the shared libraries with openat, reads them, closes them, and only at the end the listing is printed with write on the descriptor 1. The answer to the first question is therefore write, on the descriptor 1, and the program does not know whether that descriptor leads to the terminal or to a file.

In the second part, the program syscalls.c used open, write, read and close directly, and the trace showed every one of its C calls becoming exactly one system call, in the order of the source: openat with O_WRONLY|O_CREAT|O_TRUNC returned the descriptor 3, write of 14 bytes returned 14, close returned 0, the second openat with O_RDONLY returned 3 again and read returned 14 although it was allowed to read 63. The descriptor is 3 and not 0, 1 or 2 because those three are inherited from the shell as the standard input, output and error, and the kernel always gives the smallest free number; the second openat received 3 again because the file had been closed in the meantime. The trace also showed that printf is not a system call: it appeared as three write calls on the descriptor 1, which is the same mechanism seen from the other side, and that the library turns open into the more general openat with the argument AT_FDCWD.

In the third part, the option -c of strace counted the calls of one kind instead of printing them, which showed that the number of system calls follows the work asked of the kernel and not the length of the program. Overall, the work gave a practical understanding of what a system call is, of how the C library sits between the program and the kernel, and of how a running program can be observed from the outside without changing or recompiling it; all the objectives of the laboratory work were achieved.

References

1. *Gabriela Pricop. os-2026-GabrielaPricop: laboratory works of the Operating Systems course, GitHub repository*
   `https://github.com/gabriela-pricop/os-2026-GabrielaPricop`
2. *strace(1) – trace system calls and signals, Linux manual page*
   `https://man7.org/linux/man-pages/man1/strace.1.html`
3. *open(2) – open and possibly create a file, and the rule of the lowest-numbered unused descriptor, Linux manual page*
   `https://man7.org/linux/man-pages/man2/open.2.html`
4. *write(2) – write to a file descriptor, Linux manual page*
   `https://man7.org/linux/man-pages/man2/write.2.html`